



Procesamiento Distribuido II





Procesamiento de Información

A medida de que las empresas y los sistemas de información fueron evolucionando de manera tal que estos últimos dejaron de ser un mero store de data.

A partir de adquirir nuevas fuentes de información, combinarlas y analizarlas se logra generar conocimiento que se vuelve esencial a la hora de tomar decisiones y acciones para la mejora del comportamiento del sistema.



¿Cuáles son los pasos del procesamiento de la información?

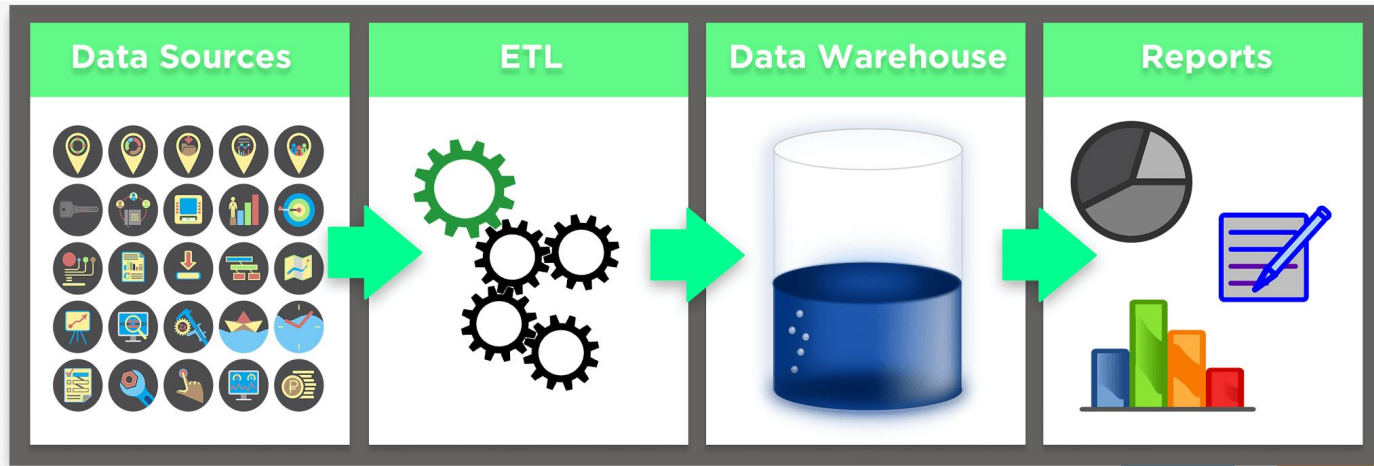




DataWarehouses y BI

Data Warehouse / OLAP / BI

A medida que los requerimientos de procesamiento se volvieron demasiado complejos para las bases de datos SQL se pasó al modelo de Data Warehouse donde el procesamiento es Batch





Hadoop

Hadoop project: componentes

Hadoop Distributed File System (HDFS™): File System distribuido capaz de contener archivo de gran volumen y provee gran rendimiento en la transmisión de la información



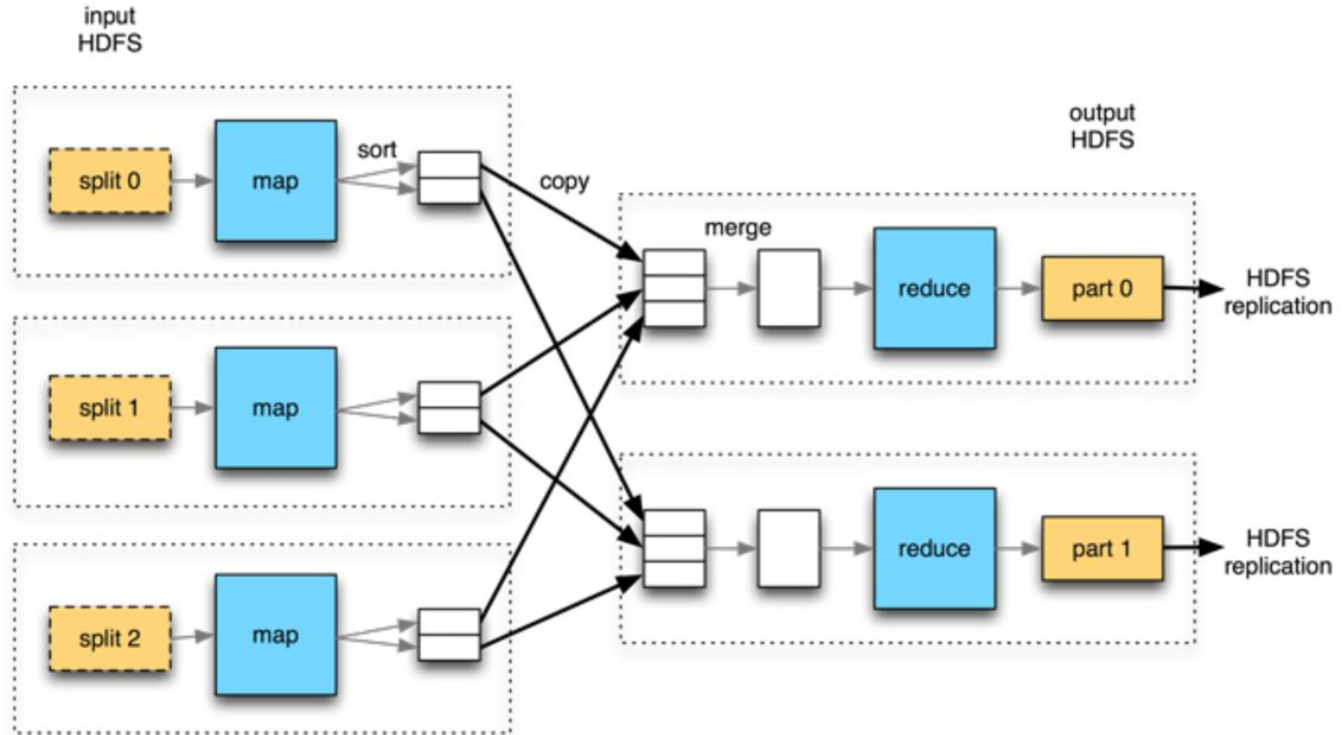
Hadoop MapReduce
Framework de procesamiento distribuido



Hadoop YARN
Framework de manejo de recursos y tareas distribuidas



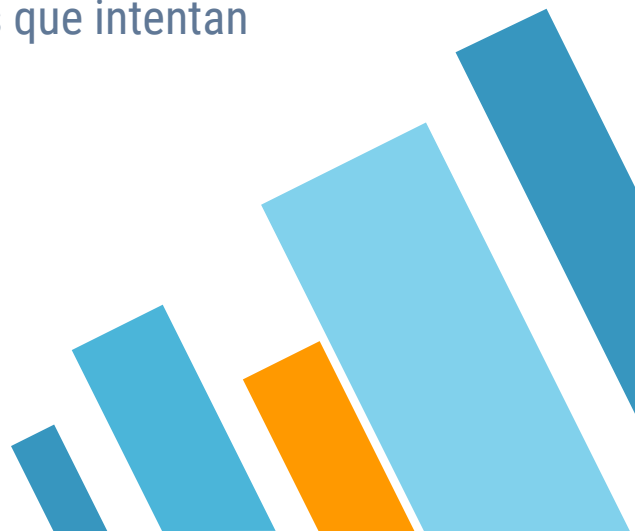
Hadoop: Map Reduce





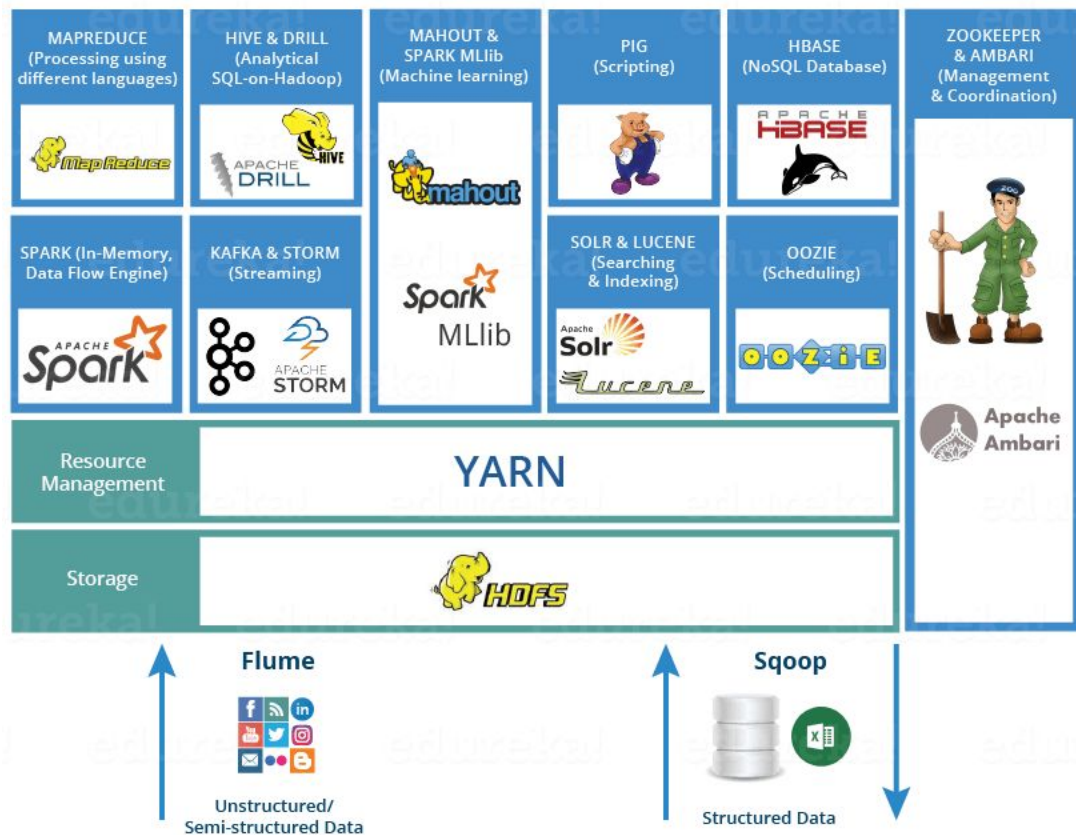
Hadoop project: Críticas

Hadoop tiene 2 grandes críticas:

1. La inherente a Map Reduce que es que hay problemas que son difíciles de modelar como un Job Map Reduce (o inclusive una cadena de Map Reduces).
 2. Al persistir cada salida intermedia a disco, se pierde mucho tiempo en IO, lo que lo hace “lento” contra otros frameworks posteriores que intentan trabajar lo más posible en memoria.
- 

Hadoop project: Ecosistema

Para facilitar el uso de Hadoop se crearon muchos proyectos satélites. Muchos tomaron vida propia y son usados por sí solos o como parte complementaria de otros frameworks.

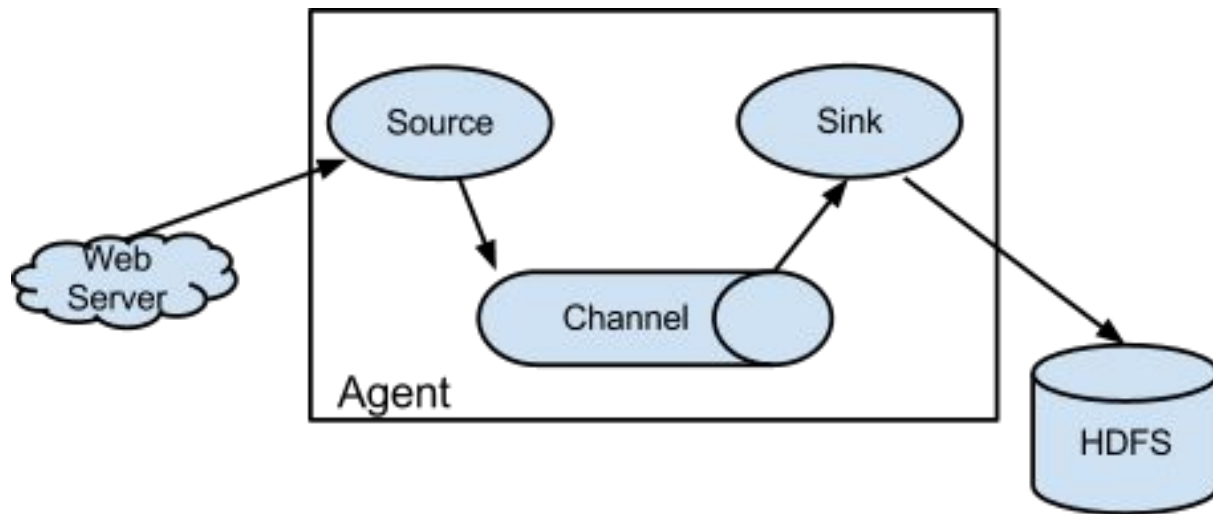




FLUME

Streaming a HDFS

Apache Flume Agent





Apache Flume Example netcat 2 HDFS

Name the components on this agent

aNet2HDFS.sources = netcat-source

aNet2HDFS.channels = memory-channel

aNet2HDFS.sinks = hdfs

Describe/configure Source

aNet2HDFS.sources.netcat-source.type = netcat

aNet2HDFS.sources.netcat-source.bind = localhost

aNet2HDFS.sources.netcat-source.port = 44445

Describe the sink

aNet2HDFS.sinks.hdfs.type = hdfs

aNet2HDFS.sinks.hdfs.hdfs.path = hdfs:/log_data/

aNet2HDFS.sinks.hdfs.hdfs.fileType = DataStream

aNet2HDFS.sinks.hdfs.hdfs.writeFormat = Text

aNet2HDFS.sinks.hdfs.hdfs.batchSize = 1000

aNet2HDFS.sinks.hdfs.hdfs.rollSize = 0

aNet2HDFS.sinks.hdfs.hdfs.rollCount = 10000

Use a channel which buffers events in memory

aNet2HDFS.channels.memory-channel.type = memory

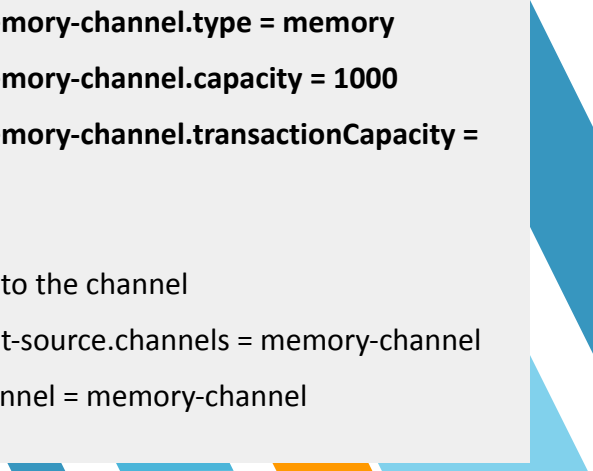
aNet2HDFS.channels.memory-channel.capacity = 1000

aNet2HDFS.channels.memory-channel.transactionCapacity = 100

Bind the source and sink to the channel

aNet2HDFS.sources.netcat-source.channels = memory-channel

aNet2HDFS.sinks.hdfs.channel = memory-channel





Pig

Pig es una plataforma de alto nivel para crear programas MapReduce utilizados en Hadoop.

Apache Pig Word count

```
input_lines = LOAD '/tmp/my-copy-of-all-pages-on-internet' AS(line: chararray);

--Extract words from each line then flatten the bag to get one word on each row
words = FOREACH input_lines GENERATE FLATTEN(TOKENIZE(line)) AS word;
--filter any words that are white spaces
filtered_words = FILTER words BY word MATCHES '\\w+';
--create a group for each word
word_groups = GROUP filtered_words BY word;
--count the entries in each group
word_count = FOREACH word_groups GENERATE COUNT(filtered_words) AS count, group AS word;
--order the records by count
ordered_word_count = ORDER word_count BY count DESC;

STORE ordered_word_count INTO '/tmp/number-of-words-on-internet';
```



Apache Pig Relational Operators

- **LOAD:** carga el contenido el file system.
- **DUMP:** Muestra en pantalla el contenido de la colección.
- **STORE:** Guarda los resultados en el file system
- **FILTER:** Selecciona las tuplas que cumplan con una condición
- **JOIN:** Une dos colecciones dado un criterio de unión.
- **ORDER:** Ordena una colección
- **DISTINCT:** Elimina los duplicados



Apache Pig Relational Operators

- **FOREACH:** Transforma cada elemento
- **GROUP:** Agrupa los elementos por criterio.
- **LIMIT:** limita la cantidad de elementos
- **UNION:** Une dos colecciones
- **SAMPLE:** Selecciona un subset de los datos de la colección de manera aleatoria.
- **SPLIT:** Parte una colección en dos dado un criterio

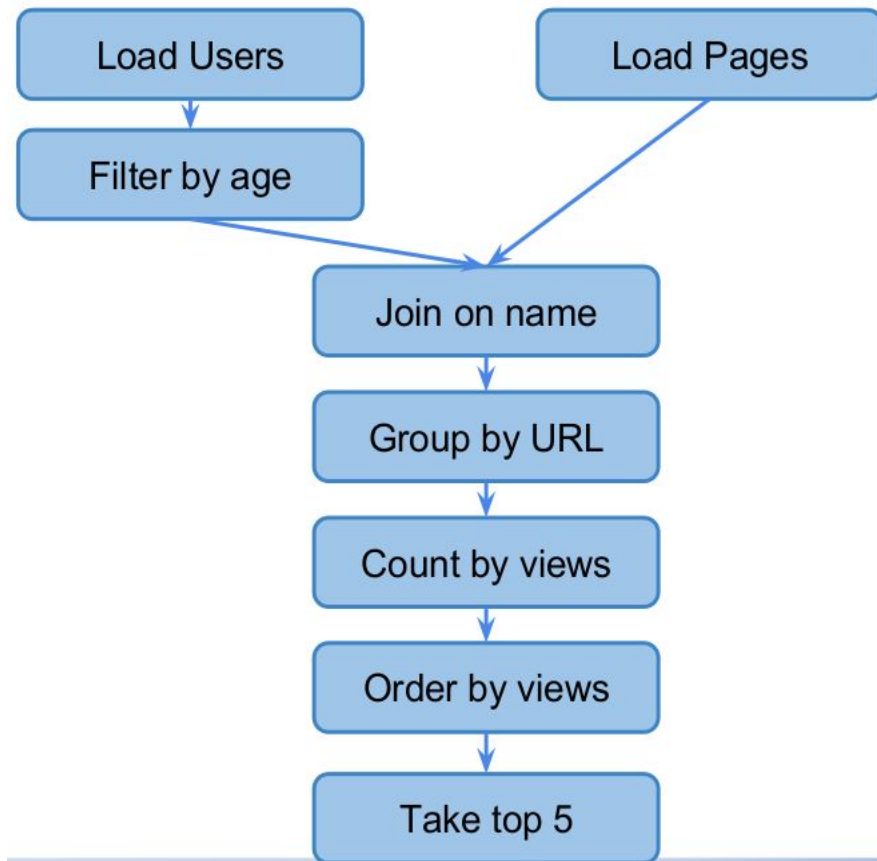
Apache Pig Example

Entrada:

- » User profiles.
- » Page visits.

Salida esperada:

- » Las 5 paginas más visitadas por usuarios con edad en el rango 18-25



Apache Pig Example

```
users = LOAD 'users' AS (name:chararray, age:int);
users_filtered = FILTER users BY age >= 18 AND age <= 25;
pages = LOAD 'pages' AS (user:chararray, url:chararray);
joined = JOIN users_filtered BY name, pages BY user;
group_by_url = GROUP joined BY url;
clicks_count_by_url = FOREACH group_by_url GENERATE GROUP, COUNT(joined) AS clicks;
sorted_counts = ORDER clicks_count_by_url BY clicks DESC;
top_5_sites = LIMIT sorted_counts 5;
--DUMP top_5_sites;
--ILLUSTRATE top_5_sites;
--DUMP top_5_sites;
STORE top_5_sites INTO 'top5sites';
```

Apache Pig User Defined Function

Java code:

```
package myudfs;

import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;

public class UPPER extends EvalFunc<String> {
    public String exec(Tuple input) throws IOException {
        if (input == null || input.size() == 0)
            return null;
        try{
            String str = (String)input.get(0);
            return str.toUpperCase();
        }catch(Exception e){
            throw new IOException("Caught exception processing input
row ", e);
        }
    }
}
```

```
REGISTER myudfs.jar;
```

```
A = LOAD 'student_data' AS (name:
chararray, age: int, gpa: float);
```

```
B = FOREACH A GENERATE myudfs.UPPER(name);
DUMP B;
```

[Pig UDF page](#)



Hive

Data warehouse construido sobre Hadoop.

Apache Hive: Word count

```
DROP TABLE IF EXISTS docs ;remove previous table
CREATE TABLE docs (line STRING); create staging table
LOAD data inpath 'input_file' overwrite INTO TABLE docs; load data into
staging table
; create word count table
CREATE TABLE word_counts AS
    SELECT    word, count(1) AS COUNT
    FROM      (SELECT Explode(Split(line, '\s')) AS word FROM      docs) temp
    GROUP BY word
    ORDER BY word;
```

Apache Hive Execution

Hive corre en el nodo master

- » Convierte la consulta HiveQL a MapReduce.
- » Submitea los jobs MapReduce jobs al cluster.

HiveQL Statements

```
SELECT zipcode, SUM(cost) AS total
FROM customers JOIN orders
ON customers.id = orders.cid
WHERE zipcode LIKE '63%'
GROUP BY zipcode
ORDER BY total DESC;
```

Hive Interpreter / Execution Engine

- Parse HiveQL
- Make optimizations
- Plan execution
- Generate MapReduce jobs
- Submit job(s) to Hadoop
- Monitor progress



MapReduce Jobs



Apache Hive Storage

Las queries en Hive se corren contra tablas, al igual que las RDBMS.

Una tabla es un directorio en HDFS que contiene uno o más archivos con el contenido de las tablas.

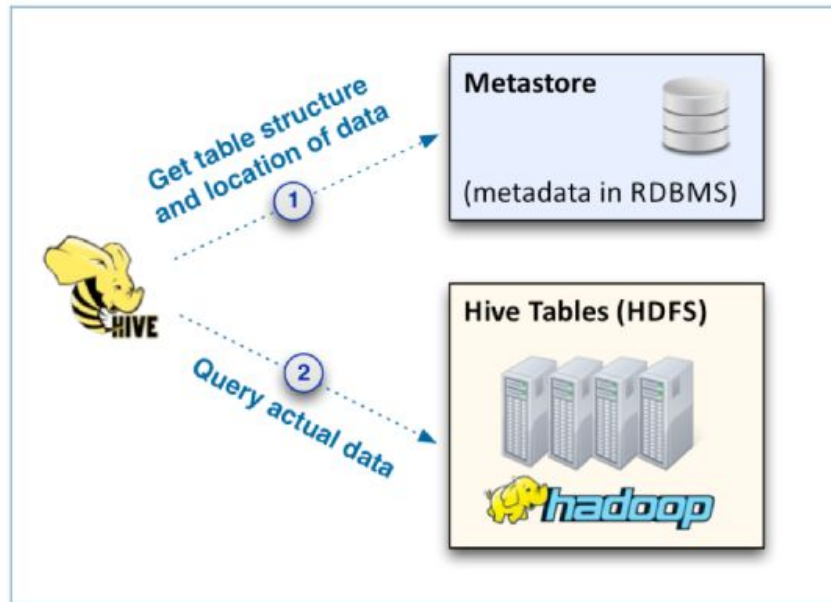
En caso de que haya particiones puede haber subdirectorios con el contenido.

Path por defecto: **`/user/hive/warehouse/<table_name>`**

El contenido se puede guardar en varios formatos (parquet, csv, json, etc.)

La información de tablas formatos y campos (**metadata**) se guarda en una base de datos relacional.

La metadata es utilizada por muchas otras herramientas



Apache Hive Create table

```
CREATE EXTERNAL TABLE IF NOT EXISTS request_logs (  
    timestamp string,  
    event_type string,  
    request_ip string,  
    request_port int  
)  
partitioned BY (year string, month string, day string)  
row format delimited fields terminated BY '\t'  
                lines terminated BY '\n'  
row format serde 'org.apache.hadoop.hive.serde2.RegexSerDe'  
    WITH serdeproperties ( "input.regex" = "([^ ]*) ([^ ]*) ([^ ]*) : ([0-9]*) $" )  
stored AS [TEXTFILE|PARQUET|ORC|RCFILE|AVRO] location  
'[s3|hdfs]://logs.company.com/request_logs/';
```

Apache Hive: Inserting Data from query

Es común usar una staging table, con la data "raw" para insertar en la tabla final.

```
INSERT [OVERWRITE|INTO] TABLE
    db.requests_logs_parquet partition (year, month, day)
SELECT timestamp,
    event_type,
    request_ip,
    request_port                year,
    printf("%04d%02d", year, month)    month,
    printf("%04d%02d%02d", year, month, day) day
FROM    staging.requests_logs;
```



Apache Hive: Querying Operands

Hive currently **supports** all common SQL-92 keywords like:

- » **SELECT**
 - » **FROM**
 - » **WHERE**
 - » **JOINS (LEFT, RIGHT, INNER, OUTER)**
 - » **UNION**
 - » **IN**
 - » **GROUP BY**
 - » **HAVING**
 - » **LIMIT**
 - » **EXISTS**
- 



Streaming



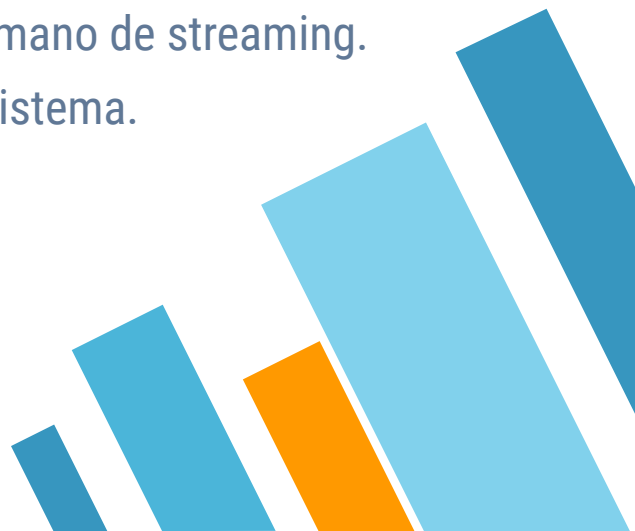
Streaming

Un stream es un flujo constante de datos que no termina y no se interrumpe.

- **Stream processing** es el procesamiento que obtiene valor de dicho stream de datos
- **Real-time processing** es un procesamiento que se realiza con tal rapidez que da sensación que los valores son procesados en tiempo real.

Aunque no son lo mismo real-time generalmente viene de la mano de streaming.

Es importante tener en cuenta que es "tiempo real" para mi sistema.





Stream ejemplos

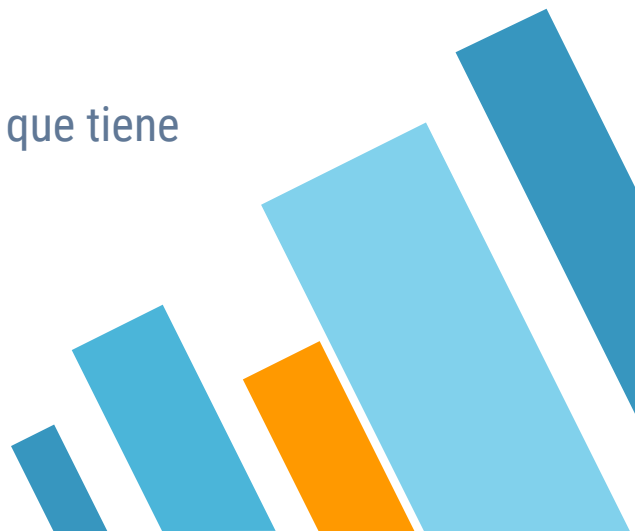
- **Web Servers:**
 - stream de clicks
 - log de visitas
 - log HTTP.
 - **Twitter:** timeline
 - **Youtube / Instagram / Facebook:** comentarios, posteos, likes, etc.
 - **Sensores:** temperatura, estado de máquinas.
 - **Celulares:** ubicación, giroscopio, eventos, etc.
- 



Evento de un Streams

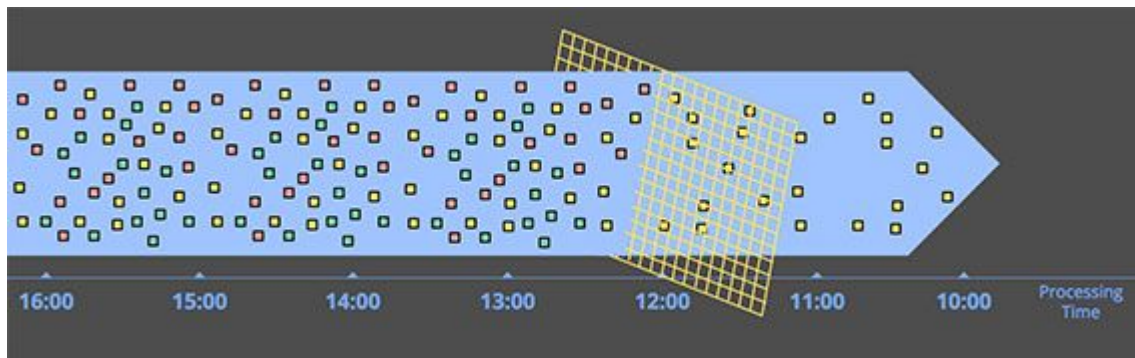
- Un **evento** es la unidad de información dentro de un **stream**.
- El armar la ingestión de información de eventos en un stream ayuda a hacer sistemas más escalables y mantenibles.
- La clave de los sistemas es enfocarse en guardar los **eventos** que modifican el sistema. No en el estado "momentáneo" del mismo.

El procesamiento en batch se puede pensar como un stream que tiene interrupciones en el medio.

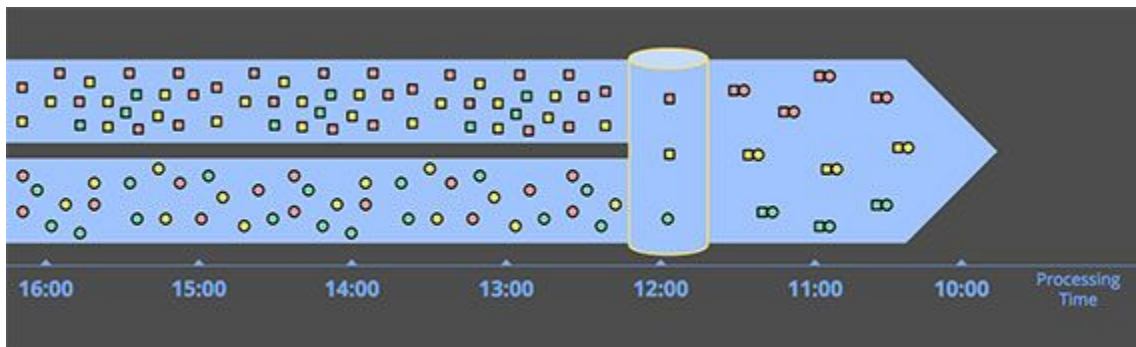


Operaciones en Streaming

➤ Filter



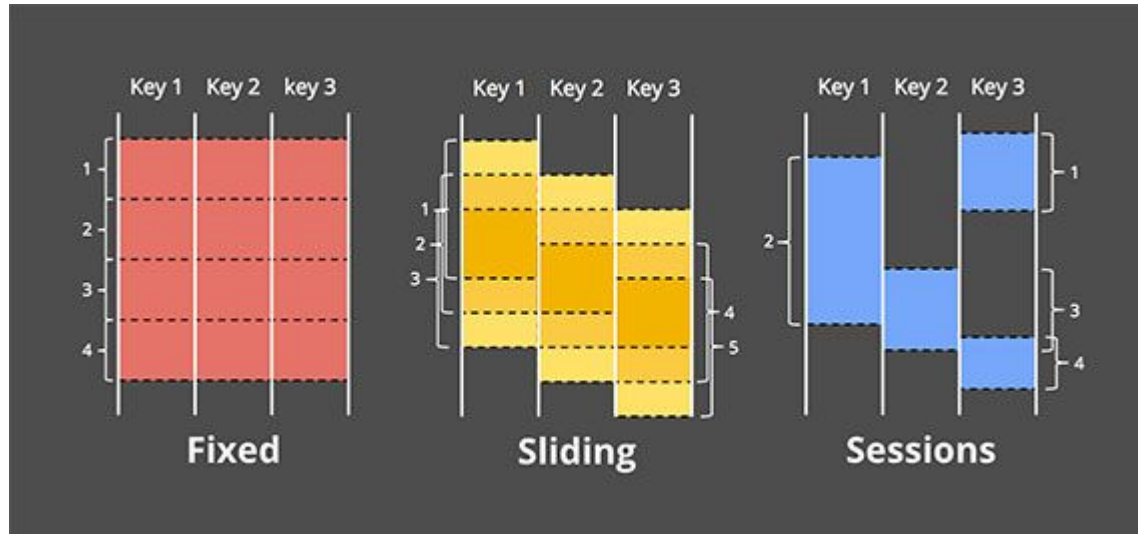
➤ Join



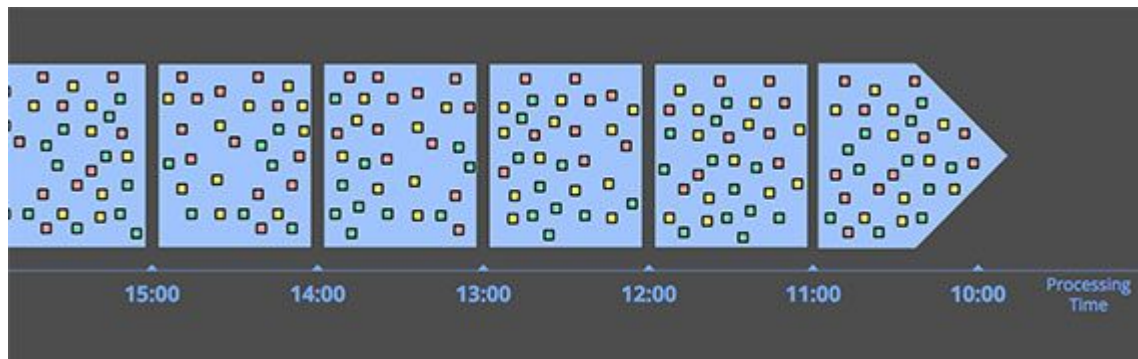
Windowing

Windowing es particionar al stream en batchs discretos.

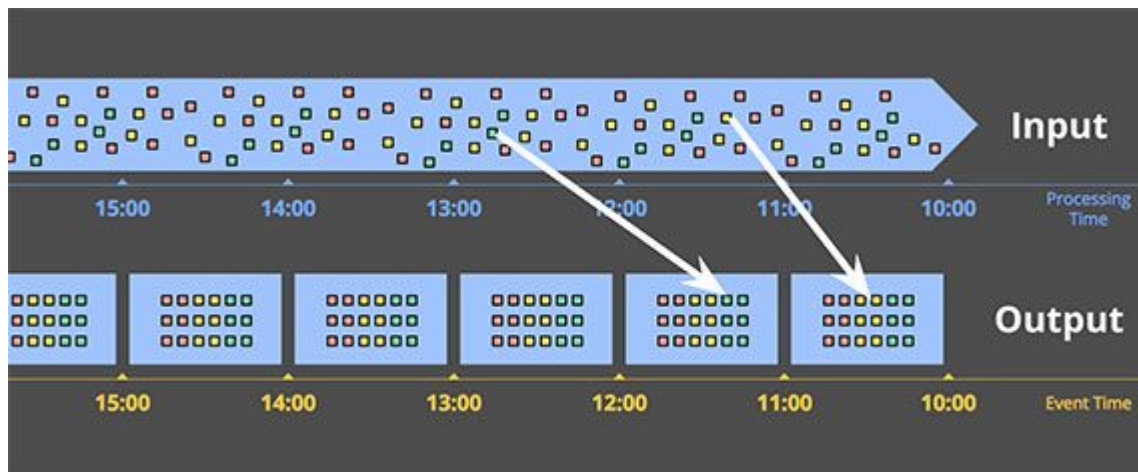
Se utiliza en casos donde el dato “global” no es útil y/o se requieren métricas por periodos (minutos, horas, etc.) o agregaciones parciales.



Processing Time



Event Time



Es común que las ventanas sean **temporales**, en cuyo caso es importante saber si podemos obtener el **tiempo de procesamiento** o se toma el **tiempo de procesamiento**.



Eventos en crudo o Información agregada

Events en crudo:

- Tiene **baja complejidad**, pero mucho volumen.
- Se puede tomar como una colección **de eventos inmutables**
- Se guardan como **fuentes de verdad**.

Información agregada:

- Se puede re-calcular de los eventos en crudo.
 - Se los considera como una **vista cacheada** sobre los mismos.
 - Permite responder rápidamente a consultas de los clientes.
- 



¿Por qué se usan los 2 tipos?

El tener 2 esquemas (lectura y escritura) trae

- **Desacoplamiento:** no hay interdependencia entre los esquemas
 - **Performance de lectura y escritura** cada esquema se mantiene óptimo para la operación que debe realizar
 - **Escalabilidad:** como la escritura es “append only” permite paralelizar y escalar la escritura de manera simple
 - **Flexibilidad:** al tener muchas “vistas cacheadas” y tener la posibilidad de recalcular se permite responder más preguntas e implementar nuevas features rápidamente
 - **Error recovery:** en caso de error se recalcula todo de nuevo luego de subsanado el error.
- 



Arquitectura de Procesamiento



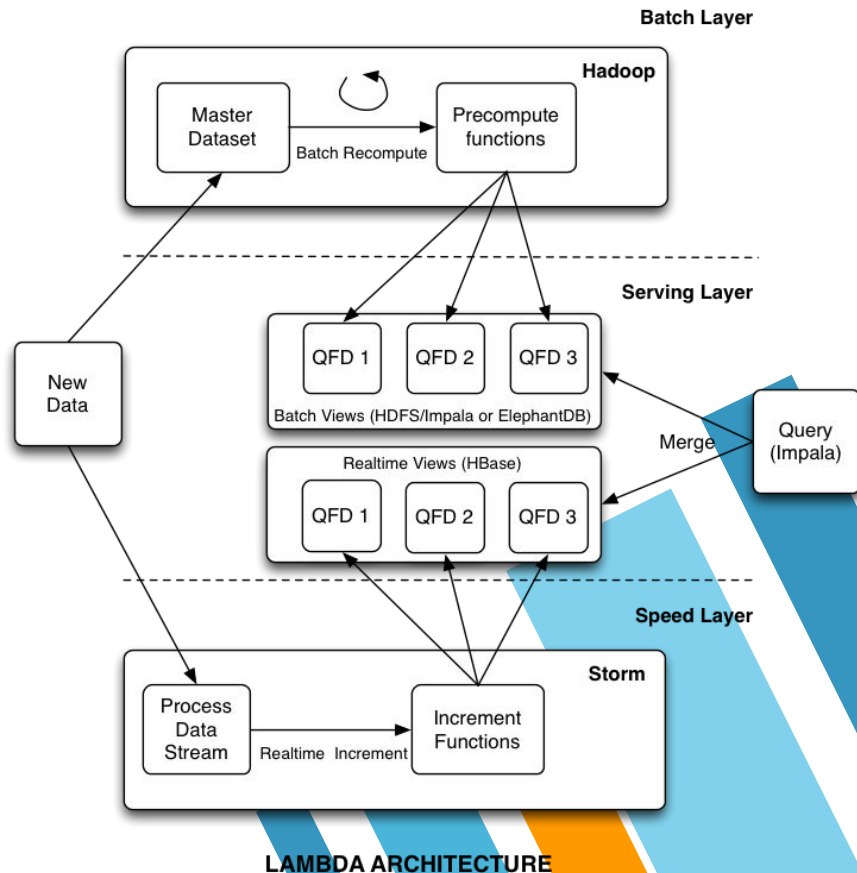
Arquitectura lambda

A medida que se popularizó Hadoop y los tiempos de procesamiento se acortaron.

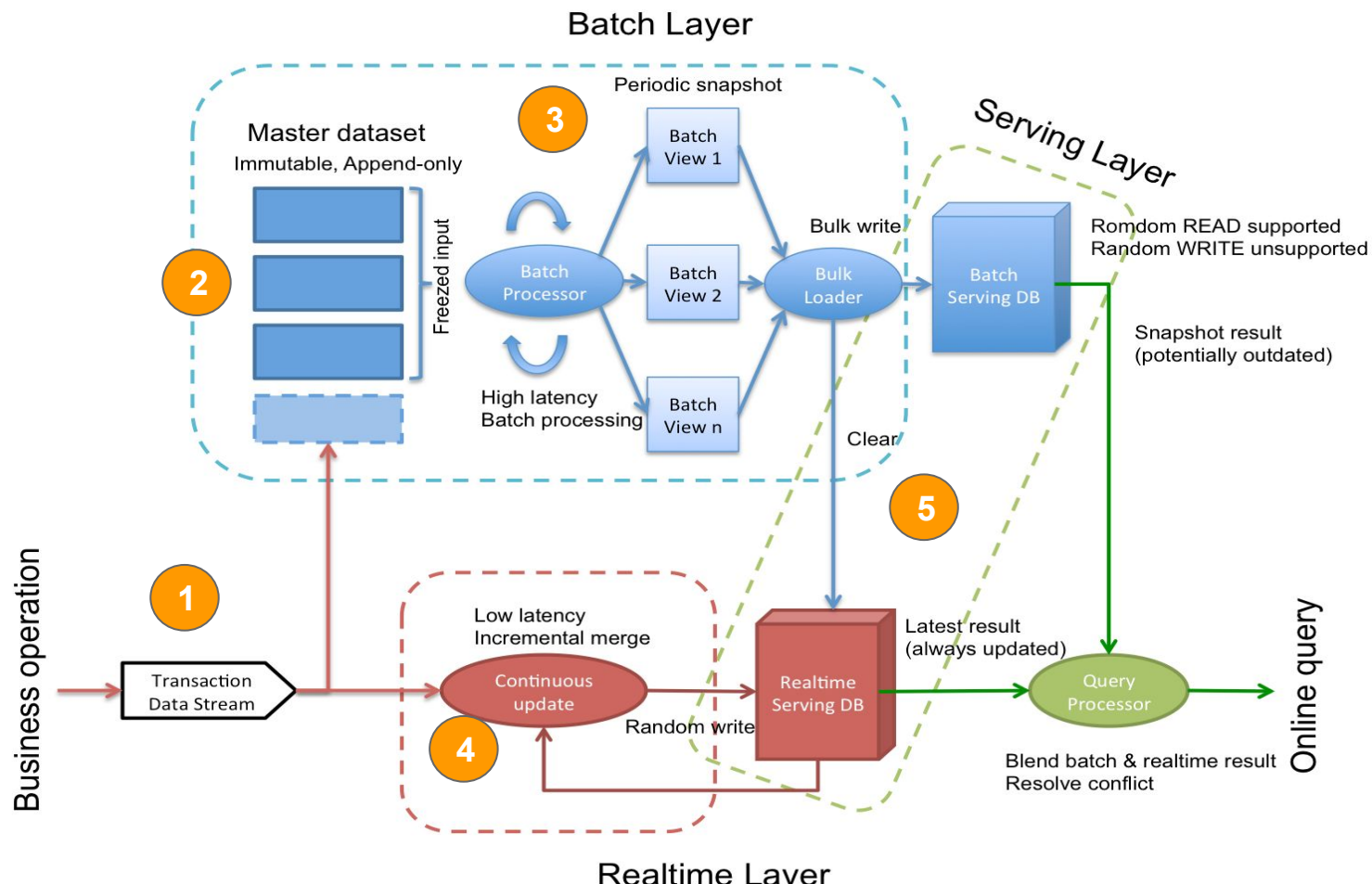
Pero surgieron:

- » **nuevas fuentes** (redes sociales, sensores)
- » **nuevos requerimientos y necesidades.**

Nathan Marz propuso la arquitectura lambda como solución a este problema.



Lambda Architecture En detaille.





Lambda Architecture

- 1 **Input Buffer:** permite soportar y bufferear el volumen de entrada mediante a colas (RabbitMQ, ActiveMQ) or logs (Kafka).
 - 2 **Master Dataset:** Colección inmutable de datos que se toma como "fuente de verdad", que permite reprocesamiento histórico.
 - 3 **Batch Layer:** Capa de procesamiento batch, que periódicamente (x hora tal vez) procesa los resultados totales hasta ese momento. Ejemplo: Hadoop.
 - 4 **Real Time Layer:** Capa de procesamiento de streaming que calcula los valores desde el último reprocesamiento hasta el momento actual. Ejemplo: Storm
 - 5 **Serving Layer:** Capa que se encarga de guardar los resultados y devolverlos a pedido de clientes. Mergeando los valores de la Batch y la Real Time
- 



Arquitectura kappa

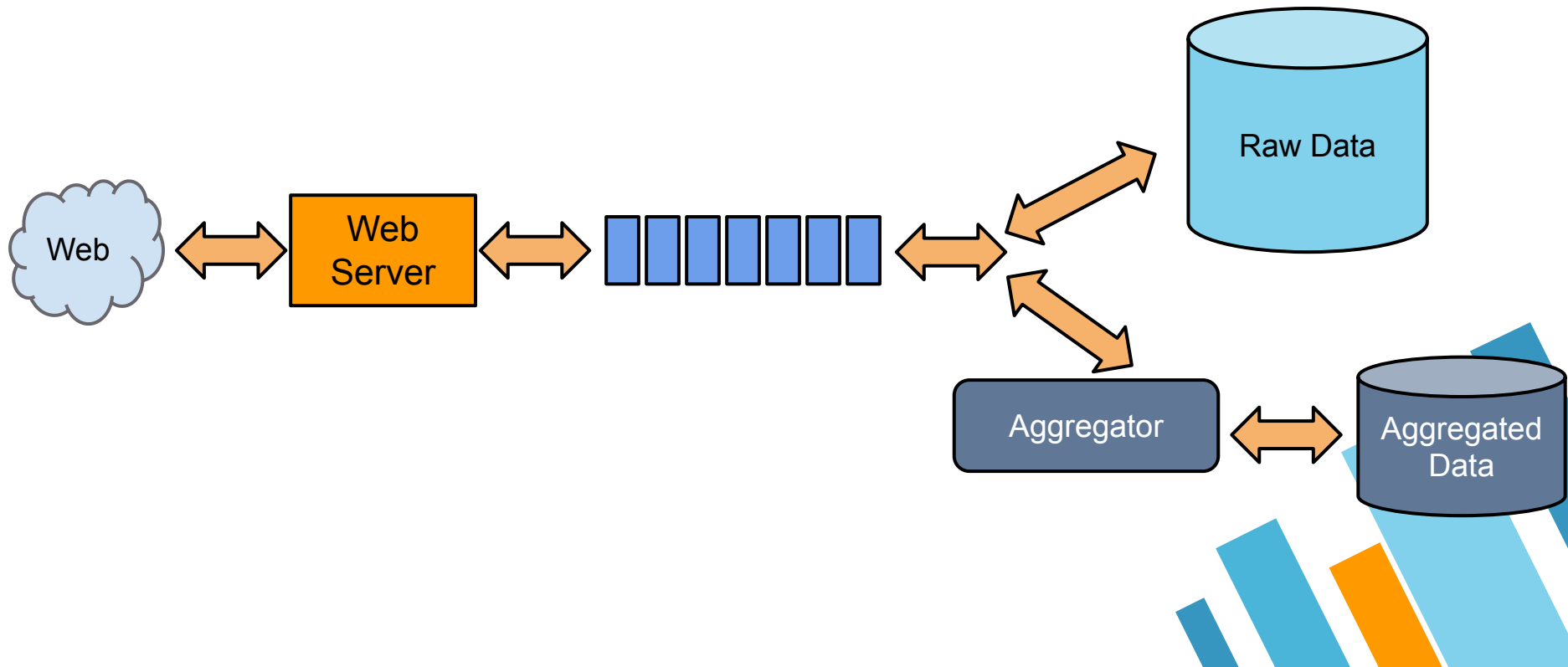
La crítica principal de la arquitectura Lambda es que obliga a tener la solución al cálculo de los valores requeridos en dos "code bases" diferentes.

Las nuevas herramientas que tienen mayor velocidad de procesamiento permiten pensar una arquitectura donde hay un solo camino de procesamiento que responde con la rapidez suficiente para que sirva para todos los casos.

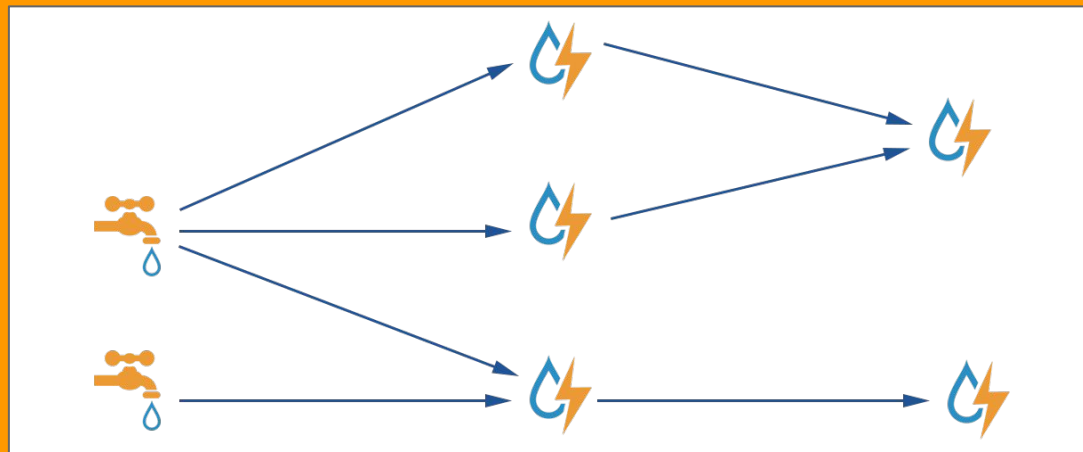
Esta arquitectura se denomina **arquitectura kappa**



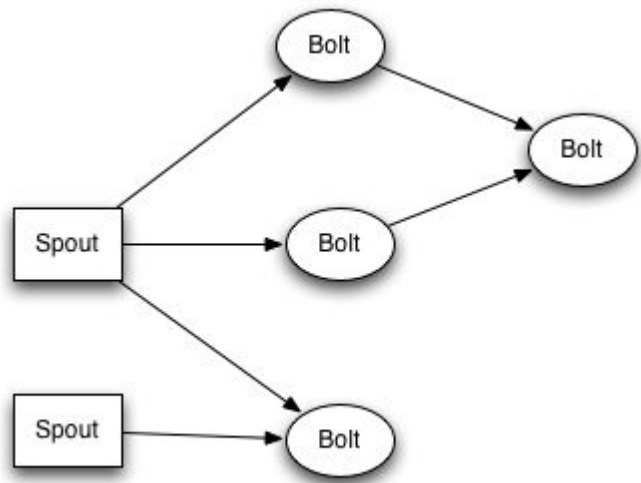
Arquitectura kappa



Apache Storm



Apache Storm Topología



- Los procesamientos en Storm se ejecutan en topologías compuestas de **“spouts”** , **“bolts”** y **streams**
- **Un spout es la fuente de datos.** O sea sabe leer y/o escuchar eventos de su fuente.
- **Un stream** representa la salida de un spout y/o bolt al cual se pueden conectar otros bolts
- **Un bolt consume de un stream transforma el mensaje y puede emitir en otro stream.**

Apache Storm Spout

```
public class TestWordSpout extends BaseRichSpout {
    SpoutOutputCollector _collector;
    public void open(Map<String, Object> conf, TopologyContext context, SpoutOutputCollector collector) {
        _collector = collector;
    }

    public void nextTuple() {
        Utils.sleep(100);
        final String[] words = new String[] {"nathan", "mike", "jackson", "golda", "bertels"};
        final Random rand = new Random();
        final String word = words[rand.nextInt(words.length)];
        _collector.emit(new Values(word));
    }

    public void declareOutputFields(OutputFieldsDeclarer declarer) {
        declarer.declare(new Fields("word"));
    }

    public void ack(Object msgId) { }
    public void fail(Object msgId) { }
    public void close() { }
}
}
```

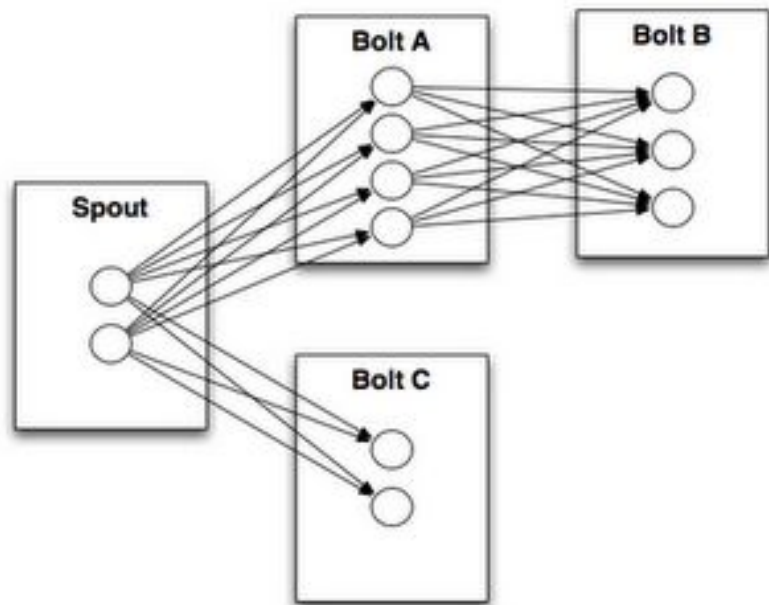
Apache Storm Bolt

```
public static class ExclamationBolt extends BaseRichBolt {  
    OutputCollector _collector;  
  
    @Override  
    public void prepare(Map conf, TopologyContext context, OutputCollector collector) {  
        _collector = collector;  
    }  
  
    @Override  
    public void execute(Tuple tuple) {  
        _collector.emit(tuple, new Values(tuple.getString(0) + "!!!"));  
        _collector.ack(tuple);  
    }  
  
    @Override  
    public void declareOutputFields(OutputFieldsDeclarer declarer) {  
        declarer.declare(new Fields("word"));  
    }  
}
```

Apache Storm Configuración

```
public static void main(String[] args) throws Exception {  
  
    Config conf = new Config();  
    conf.setNumWorkers(2); // use two worker processes  
  
    TopologyBuilder builder = new TopologyBuilder();  
    builder.setSpout("words", new TestWordSpout(), 10);  
    builder.setBolt("exclaim1", new ExclamationBolt(), 3).shuffleGrouping("words");  
    builder.setBolt("exclaim2", new ExclamationBolt(), 2).shuffleGrouping("exclaim1");  
  
    boolean runLocal = shouldRunLocal();  
    if(runLocal){  
        LocalCluster cluster = new LocalCluster();  
        cluster.submitTopology("test-topology", conf, builder.createTopology());  
    } else {  
        StormSubmitter.submitTopology("test-topology", conf, builder.createTopology());  
    }  
}
```

Apache Storm Stream groupings



Al registrarse con un stream el bolt indica el tipo distribución que tendrá los eventos para controlar cómo le llega a los workers del mismo estos eventos.

Algunos de los groupings posibles son:

- » **Shuffle grouping**
- » **Fields grouping**
- » **Partial Key grouping**
- » **All grouping**
- » **Global grouping:**



Apache Storm Stream Grouping

```
TopologyBuilder builder = new TopologyBuilder();

builder.setSpout("sentences", new RandomSentenceSpout(), 5);
builder.setBolt("split", new SplitSentence(), 8)
    .shuffleGrouping("sentences");
builder.setBolt("count", new WordCount(), 12)
    .fieldsGrouping("split", new Fields("word"));
```





Spark



Spark Word Count

```
val sc = new SparkContext(new SparkConf().setAppName("Spark  
Count"))
```

Context

Resilient distributed
datasets (RDDs)

```
// read in text file and  
val lines = sc.textFile("path")
```

```
// split each document into words  
val words = lines.flatMap(_.split(" ")).map((_, 1))
```

Transformation

```
//count the occurrence of each word
```

```
val wordCounts = words.reduceByKey(_ + _)
```

Action

SparkSQL

```
case class Person(name: String, age: Long) // you can use custom classes
```

```
// Create an RDD of Person objects from a text file, convert it to a Dataframe
```

```
val peopleDF = spark.sparkContext
```

```
.textFile("examples/src/main/resources/people.txt")
```

```
.map(_._split(",")) .map(attributes => Person(attributes(0), attributes(1).trim.toInt)) .toDF()
```

```
peopleDF.createOrReplaceTempView("people") // Register the DataFrame as a temporary view
```

```
// SQL statements can be run by using the sql methods provided by Spark
```

```
val teenagersDF = spark.sql("SELECT name, age FROM people WHERE age BETWEEN 13 AND 19")
```

```
teenagersDF.map(teenager => "Name: " + teenager(0)).show()
```

```
teenagersDF.map(teenager => "Name: " + teenager.getAs[String]("name")).show() // or by field
```

name

Spark Streaming Example

```
// Create a local StreamingContext with two working thread and batch interval of 1 second.
val conf = new SparkConf().setMaster("local[2]").setAppName("NetworkWordCount")
val ssc = new StreamingContext(conf, Seconds(1))

// Create a DStream that will connect to hostname:port, like localhost:9999
val lines = ssc.socketTextStream("localhost", 9999)

val words = lines.flatMap(_.split(" ")) // Split each line into words

// Count each word in each batch
val pairs = words.map(word => (word, 1))
val wordCounts = pairs.reduceByKey(_ + _)

wordCounts.print() // Print the first ten elements of each RDD generated in this DStream to the console

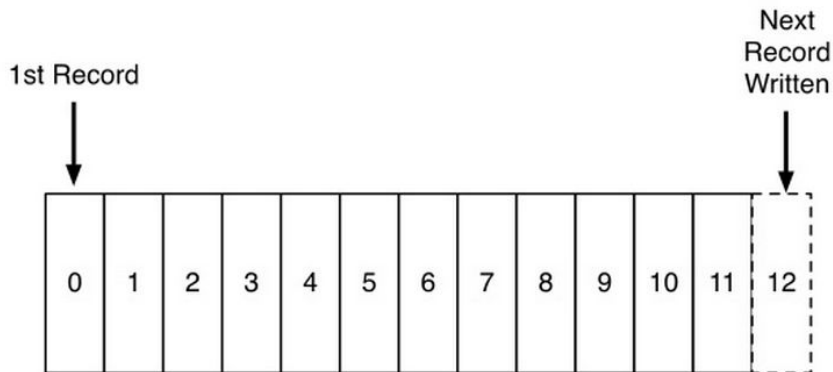
ssc.start()           // Start the computation
ssc.awaitTermination() // Wait for the computation to terminate
```



Log - Kafka

The Log

- **Secuencia de eventos ordenada, append-only**
- Cada evento se agrega al final y se le asigna un número de secuencia.
- Esa secuencia da una noción de “temporalidad” a cada evento.
- La lectura se realiza por cliente, que mantiene el número de secuencia correspondiente al último evento leído.
- Los eventos no se borran (en principio) por lo que se puede recomenzar el proceso





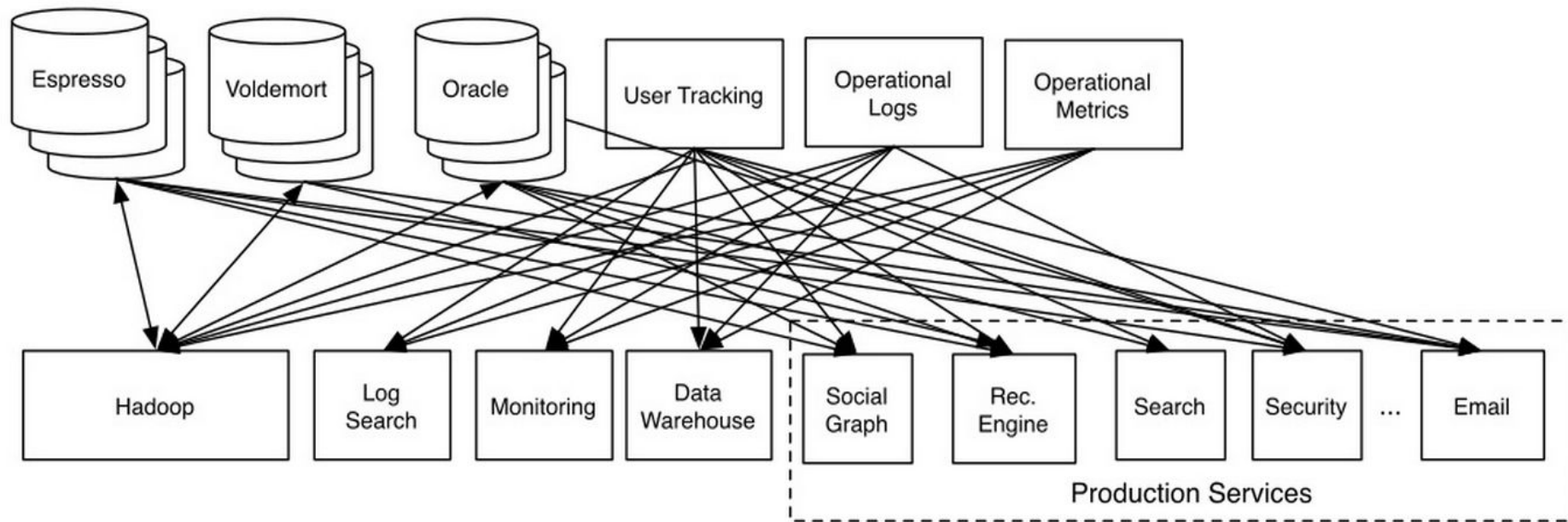
The Log: Integración de los eventos

Muchas organizaciones comenzaron a utilizar muchas herramientas para guardar y procesar sus datos (Hadoop, Hbase, Spark, MongoDB, etc).

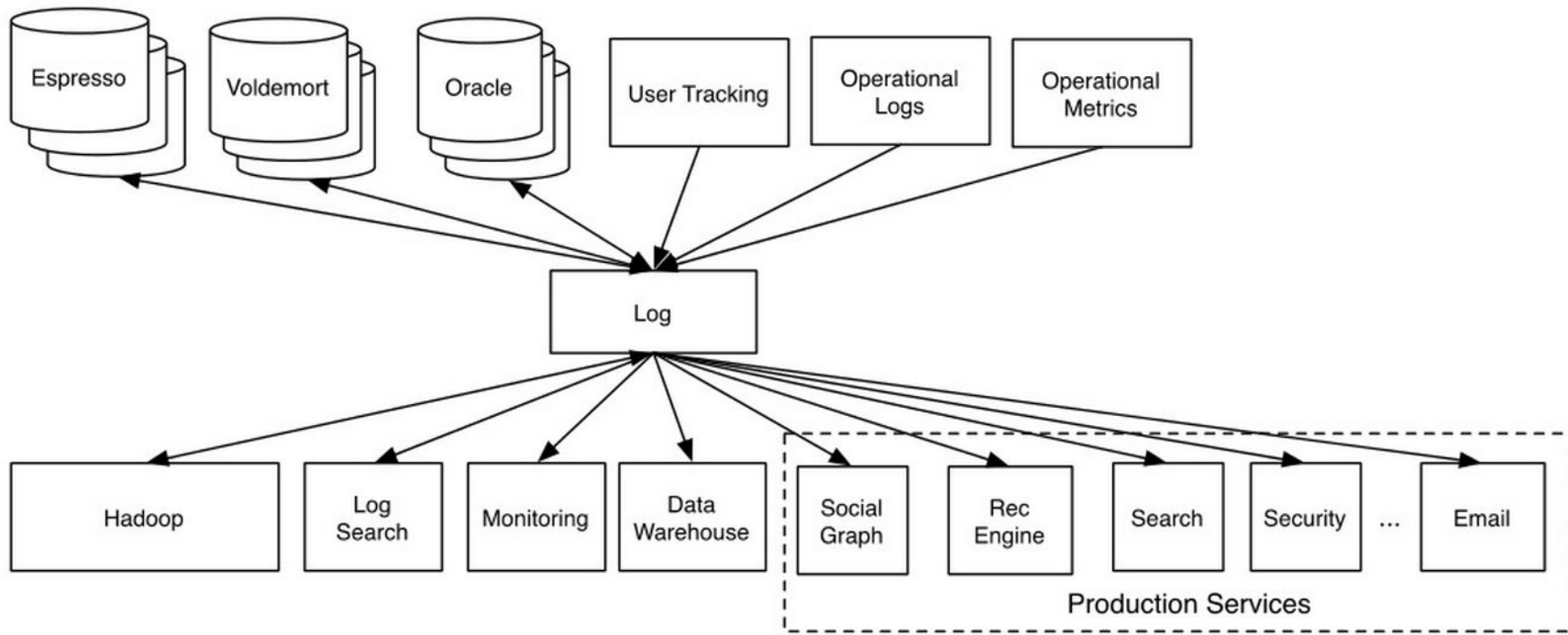
Un sistema de log persistente que centralice los eventos crudos para que todos los demás los consuman permite:

- Darle un tiempo lógico a cada evento único (processing time) para todos los procesamientos posteriores.
 - **La producción de los datos se vuelve disociada del consumo.** El que adquiere pública en el log y el que consume lo levanta cuando puede a medida que su carga lo permita
 - **El sistema que consume solo sabe del log:** No tiene que tener dependencias al que adquiere y produce.
 - **La única dependencia entre sistemas es el formato del evento.**
- 

The Log: LinkedIn's before infrastructure



The Log: LinkedIn's after infrastructure



Apache Kafka: The Log for Big Data

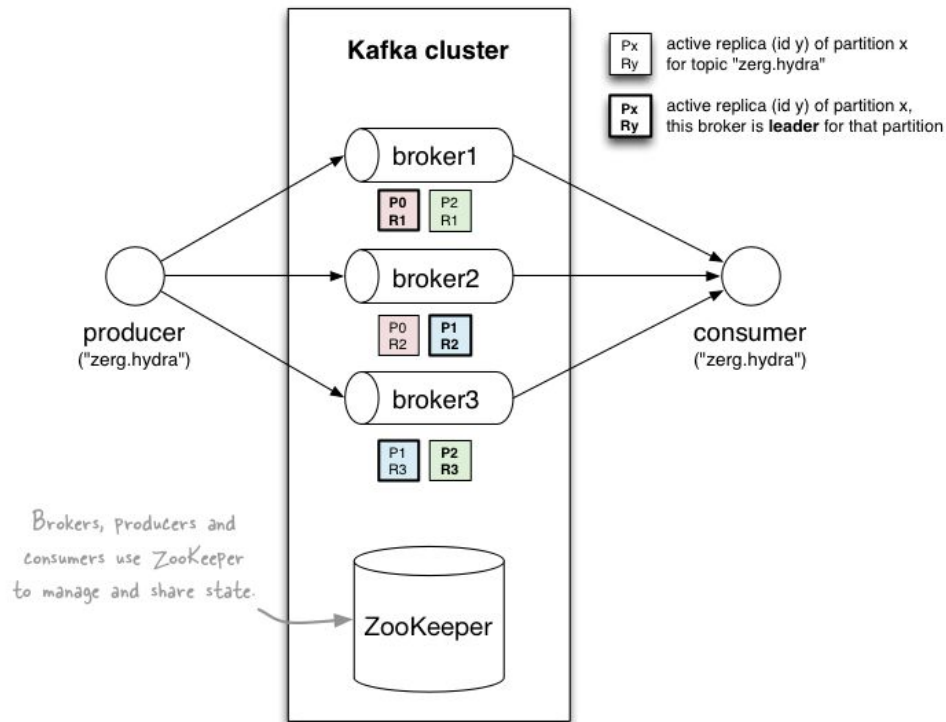


Kafka es un sistema de mensajería diseñado para ser rápido, escalable y durable. Más allá del desacoplamiento de sistemas, se utiliza el principio como parte inicial de las cadenas de procesamiento a manera de buffer persistente.

- » **Kafka + (Samza, Spark or Storm)**
- » **Flume or Logstash + Kafka**
- » **Kafka + Flume**
- » **Kafka + custom apps.**
- » **Kafka + Kafka Connectors and Kafka + Kafka Stream.**

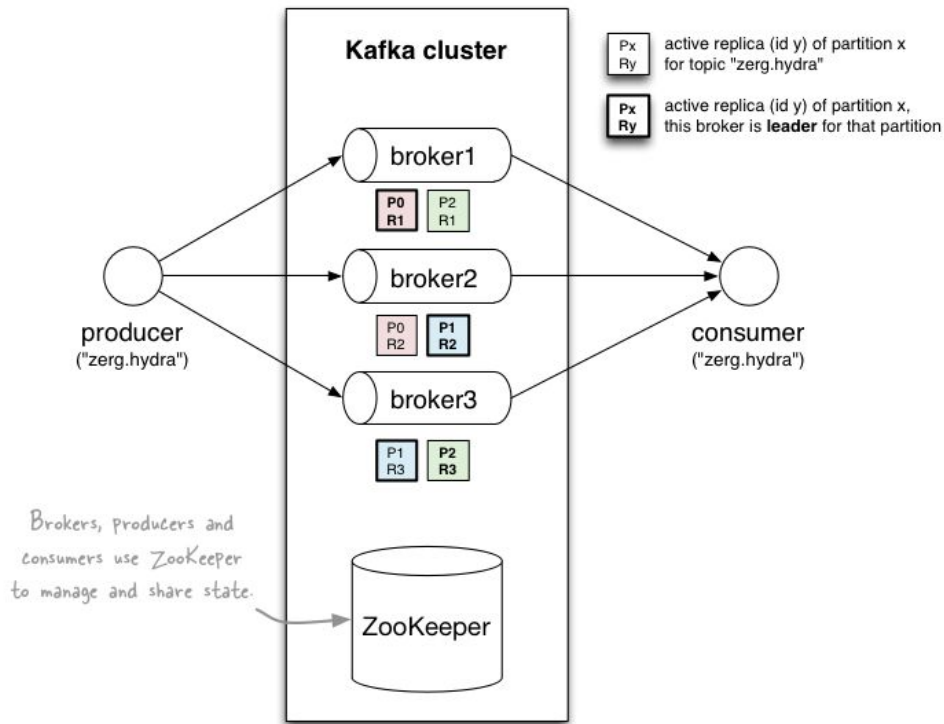
Apache Kafka: Conceptos

- **Los brokers:** son los nodos del sistema, existe uno por nodo lógico del cluster.
- **Los tópicos:** son conjuntos lógicos de mensajes (eventos) que se quieren transmitir.



Apache Kafka: Conceptos

- **Los productores** son los clientes que quieren escribir mensajes en el log. Escriben una partición de un tópico dentro de un broker.
- **Los mensajes** dentro de una partición de un tópico están ordenados por el momento de escritura.
- **Los consumidores** son los clientes que leen, se les asigna una partición, la leen completa antes de seguir con otras.





Apache Kafka: Conceptos

- En principio kafka persiste todos los mensajes en disco. Aunque esto muchas veces se evita por una cuestión de espacio.
 - Un mensaje es un par clave, valor ambos guardados como un conjunto de bytes (sin formato aparente)
 - Los mensajes se asignan a una partición por el productor (generalmente a partir de un cálculo sobre la clave)
- 



Apache Kafka: Conceptos

- Los consumidores pueden elegir recordar cuál fue el último mensaje leído (offset) o que lo maneje kafka). Se puede empezar a leer de nuevo.
 - Se puede habilitar **compaction**, que hace que solo quede el último mensaje de cada clave (si no interesa la historia)
 - La cantidad de particiones en un tópico determina el paralelismos (cuantos productores y consumidores pueden estar conectados al mismo tiempo).
 - Cada productor/consumidor se conecta a una partición dentro del tópico.
 - Dentro de esa partición se garantiza el orden
- 



Esquemas en Apache Kafka

Kafka guarda los mensajes de manera raw, no hay esquemas por defecto. Usualmente hay un código compartido entre los que leen y escriben en el tópic. Existen dos maneras de trabajar mensajes con esquema.

- **Embebido:** el esquema va en el mensaje.
- **Centralizado:** Se crea un servidor central que tiene los esquemas. Kafka provee uno, si se lo requiere.

Idealmente se debería soportar que los esquemas cambien de manera evolutiva sin que afecte a los productores y consumidores.





Dmitriy Ryaboy

@squarecog



Follow

Schemas are an impediment to moving fast the same way street lights are. Useless if you are alone, critical when there's a 100 of you.



RETWEETS

110

FAVORITES

110



12:21 PM - 21 Aug 2014



Kafka Spring Código

Ejercicio 1 - Consumo / Producción

1. Generar un proyecto con spring para mostrar el consumo y producción de mensaje



Kafka - Proyecto

Para generar el proyecto vamos a utilizar el [inicializador de spring](#) con

- Maven
 - Spring Web
 - Kafka
- 



Kafka Docker

Para correr kafka necesitamos un zookeeper además de kafka, una manera fácil de obtener ambos servicios es utilizar docker-compose.

Bajar de campus e incorporar el archivo docker-compose.yml.





Kafka / Spring

Spring provee una clase que engloba los accesos a kafka llamada `KafkaTemplate` la cual se instancia como bean y se puede inyectar para utilizar en los servicios

Para la conexión se precisa bajar al proyecto el `application.yml`



Kafka Producer

El producer utiliza el template para enviar mensajes a un t3pico.

```
@Service
public class Producer {
    private static final Logger logger = LoggerFactory.getLogger(Producer.class);
    private static final String TOPIC = "message";
    private final KafkaTemplate<String, String> kafkaTemplate;

    @Autowired
    public Producer(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void sendMessage(String message) {
        logger.info(String.format("#### -> Producing message -> %s", message));
        this.kafkaTemplate.send(TOPIC, message);
    }
}
```

Kafka Consumer

El consumer se define mediante a un objeto que se anota con `KafkaListener` y es llamado cada vez que se recibe un mensaje.

```
@Service
public
    class Consumer {
        private final Logger logger = LoggerFactory.getLogger(Consumer.class);

        @KafkaListener(topics = "message", groupId = "group_id")
        public void consume(String message) throws IOException {
            logger.info(String.format("#### -> Consumed message -> %s", message));
        }
    }
```


Controller

Para poder probarlo utilizamos un controller para mandar mensajes via HTTP

```
@RestController
@RequestMapping(value = "/kafka")
public class Controller {

    private final Producer producer;

    @Autowired
    public Controller(Producer producer) {
        this.producer = producer;
    }

    @PostMapping(value = "/publish")
    public void sendMessageToKafkaTopic(@RequestParam("message") String message) {
        this.producer.sendMessage(message);
    }
}
```

Ejercicio 2 - Consumo / Producción JSON

1. Cambiar los productores y consumidores para mandar un objeto User con nombre y edad.



Kafka Streams



Kafka Streams

Kafka Streams es una **librería de procesamiento de información** provisto por kafka.

Mediante la librería se define un pipeline de operaciones a realizar, se la envía al cluster y cada nodo procesa el mismo contra la información que tiene dentro de sus particiones.

Provee

- Fault-tolerance
 - Procesamiento por mensaje
 - primitivas de streaming.
- 

Kafka Streams WordCount

```
public class WordCountDemo {  
    public static void main(String[] args) throws Exception {  
        Properties props = new Properties();  
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "streams-wordcount");  
        Pattern pattern = Pattern.compile("\\W+", Pattern.UNICODE_CHARACTER_CLASS);  
  
        KStreamBuilder builder = new KStreamBuilder();  
        KStream<String, String> source = builder.stream("streams-file-input");  
  
        KStream<String, Long> wordCounts = source  
            .flatMapValues(value-> Arrays.asList(pattern.split(value.toLowerCase())))  
            .map((key, word) -> new KeyValue<>(word, word))  
            .countByKey("Counts")  
            .toStream();  
    }  
}
```



Kafka Streams WordCount

```
counts.to(Serdes.String(), Serdes.Long(), "streams-wordcount-output");

KafkaStreams streams = new KafkaStreams(builder, props);
streams.start();

// usually the stream application would be running forever,
// in this example we just let it run for some time and stop since the input data is finite.
Thread.sleep(5000L);

streams.close();
}
```





Cloud processing

Además de la posibilidad de deployar cualquiera de estos frameworks en soporte cloud, los proveedores ofrecen servicios de procesamiento, que permite indicar el código a ejecutar y los datos sobre los cuales correr y pagar por el tiempo de procesamiento.

Algunos de estos servicios son:

- » [Amazon lambda](#)
 - » [Google Cloud Functions](#)
 - » [Azure Functions](#)
- 



References

- » <http://radar.oreilly.com/2015/08/the-world-beyond-batch-streaming-101.html>
 - » <https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>
 - » <http://blog.confluent.io/2015/01/29/making-sense-of-stream-processing/>
 - » <http://blog.confluent.io/2015/02/25/stream-data-platform-1/>
 - » <http://blog.confluent.io/2015/02/25/stream-data-platform-2/>
 - » <http://blog.confluent.io/2015/03/04/turning-the-database-inside-out-with-apache-samza/>
 - » <http://www.cakesolutions.net/teamblogs/comparison-of-apache-stream-processing-frameworks-part-1>
- 



References

- » <http://www.michael-noll.com/blog/2014/08/18/apache-kafka-training-deck-and-tutorial/>
 - » <http://blog.cloudera.com/blog/2014/09/apache-kafka-for-beginners/>
 - » <http://samza.apache.org/learn/documentation/latest/comparisons/introduction.html>
 - » <https://storm.apache.org/documentation/Tutorial.html>
 - » <http://www.slideshare.net/gwenshap/kafka-for-dbas>
- 



CREDITS

Content of the slides:

- » POD - ITBA - 2022

Images:

- » POD - ITBA - 2022
- » obtained from: commons.wikimedia.org

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by [Unsplash](https://unsplash.com)
- 